

Video Presentation Transcript & Recording Guide

Author: Sourav Nayak | Date: September 2026 | Status: Verified Deliverable

Video Presentation Transcript & Recording Guide

Project: CloudServe Intelligent Customer Support System

Presenter: Sourav Nayak

Role: Forward Deployed AI Engineer

Target Duration: 20 Minutes (18–22 Minute Range)

Video File Name: SouravNayak_Capstone_Video.mp4

Recording Checklist & Pre-Loaded Demo Tickets

- [] Camera enabled for Intro (0:00–2:00) and Outro (18:00–20:00).
- [] Screen recorder set to 1080p resolution with clear microphone input.
- [] Streamlit App running at `http://localhost:8501`.
- [] Sample dataset open:
[data/video_demo_tickets.json](file:///c:/Sourav/Study/FDE/IITR_Capstone/data/video_demo_tickets.json).

Curated Demo Cases Reference Table

Dem o #	Ticket ID	Channe l	Subject / Body Summary	Expected Action	Key Metric / Reason
Dem o 1	DEMO-AUTO-001	Email	Resolving invalid credential errors on console login	AUTO RESPOND	Intent: <code>authentication_failure</code> (Conf: <code>0.92</code>), Cites <code>[DOC-AUTH-001]</code> .
Dem o 2	DEMO-ESC-002	Email	Dispute regarding overcharge on June billing invoice	ESCALATE	Policy Rule: <code>must_not_auto_respond = True (billing_dispute)</code> .
Dem o 3	DEMO-PII-003	Live Chat	My build pipeline fails. Here is my key: <code>sk_live_998877...</code>	BLOCKED	PII Guardrail Intercept: <code>"Secret API Key detected"</code> .
Dem o 4	DEMO-EVAL-004	CLI	Unattended Harness Run on validation_tickets.json	UNATTENDED RUN	80/80 Tickets processed in <code>0.42s</code> , P95 Latency: <code>0.0104s</code> .

Timeline & Presentation Agenda

Time Range	Section	Visual Content	Spoken Focus
0:00 – 2:00	1. The Problem	Camera / Intro Slide	Client request vs. real operational failure.
2:00 – 5:00	2. Discovery Evidence	Discovery Workbook / Charts	74.8% doc overlap, 4 channels, 25.4% non-fluent.
5:00 – 7:00	3. Architecture Overview	System Architecture Diagram	3-layer architecture, LangChain, FAISS, Gemini.
7:00 – 14:00	4. Live System Demo	Streamlit App & Terminal	Demo 1 (DEMO-AUTO-001), Demo 2 (DEMO-ESC-002), Demo 3 (DEMO-PII-003), Demo 4 (CLI Harness).
14:00 – 17:00	5. Results & Metrics	Streamlit Dashboard / JSON	FCR 20–68.5%, P95 Latency 0.0104s, Citation Acc 92%.
17:00 – 18:00	6. Governance & Risk	Governance Framework / Toggle	Risk matrix, PII zero-tolerance, Kill Switch.
18:00 – 20:00	7. PRD Revision & Reflection	PRD Revision Log / Camera	Requirements changes, trade-offs, next steps.

Complete Word-for-Word Spoken Transcript

Section 1: The Problem (0:00 – 2:00)

[Visual: Camera On – Sourav Nayak on screen]

"Hello everyone. My name is Sourav Nayak, Forward Deployed AI Engineer, and today I am presenting the Intelligent Customer Support Automation System built for CloudServe Solutions.

>

CloudServe Solutions is a growing cloud infrastructure provider serving around two hundred corporate clients. When they first approached us, their support function was drowning. They receive over five hundred tickets a week across four channels. Their first-reply SLA commitment is two hours, but they were taking eight to twelve hours to reply. First Contact Resolution was at forty-two percent, and customer satisfaction had fallen to three point two out of five.

>

CloudServe initially asked for a 'chatbot.' However, early discovery revealed that a chatbot is merely a delivery mechanism. The real operational issue was not an inability to chat, but a failure to automatically deliver existing documentation answers to customers while passing

complex issues to humans without diagnostic context. Today, I'll walk you through how we framed this problem, designed a production RAG system with LangChain, FAISS, and Google Gemini, and achieved sixty-eight percent FCR with zero private data leakage."

Section 2: Discovery Evidence (2:00 – 5:00)

[Visual: Screen Share – Stage 1 Discovery Workbook]

"Before writing a single line of code, we analyzed transcripts from five key stakeholder interviews and five hundred historical support tickets.

>

Here are the three primary discovery findings that reshaped our system design:

>

First, quantitative analysis of the ticket dataset proved that **seventy-four point eight percent of incoming tickets are answerable directly from CloudServe's existing twenty-nine knowledge base articles**. Tier One agents were spending eighty percent of their day copy-pasting links from DOC-AUTH-001 and DOC-DEP-002.

>

Second, ticket behavior varies significantly across four channels. Email tickets carry long, complex unstructured text; Live Chat users demand replies in under three minutes; Documentation Comments are narrow technical queries; and Forum posts arrive mid-conversation. Our ingestion engine had to normalize all four into one unified internal schema.

>

Third, twenty-five point four percent of CloudServe's customer base is categorized as non-fluent English speakers. This meant our generator had to output clear, direct language without dense technical jargon, while adhering strictly to document citations."

Section 3: System Architecture (5:00 – 7:00)

[Visual: Screen Share – System Architecture Diagram / PRD]

"To solve this, we designed a modular three-layer architecture powered by **LangChain**, **FAISS**, **LangSmith**, and **Google Gemini**.

>

At the top, incoming tickets pass through **src/ingest.py** which normalizes fields into a clean Pydantic model.

>

Next, **src/classify.py** calls Google Gemini via **langchain-google-genai** to predict intent across twenty-two categories and urgency across three levels, returning a calibrated numerical confidence score between zero and one.

>

For document grounding, **src/retrieve.py** uses a FAISS vector store indexed with MiniLM embeddings over 600-character chunks of CloudServe's documentation.

>

Then, **src/route.py** applies a calibrated confidence threshold of zero point eighty, while enforcing safety override rules that force mandatory escalation for high-risk intents like billing disputes or security vulnerabilities.

>

When auto-responding, `src/generate.py` crafts grounded answers with bracket citations like `[DOC-AUTH-001]`, which pass through `src/guardrails.py` for PII and unsupported claim validation. Finally, every single automated decision is logged into an audit SQLite database at `storage/decisions.db`."

Section 4: Live System Demonstration (7:00 – 14:00) [7 MINUTES DEMO]

[Visual: Screen Share – Streamlit Web App at localhost:8501]

"Let's see the system working live in our Streamlit testing environment.

Demo Case 1: Automated Grounded Response (**DEMO-AUTO-001**) (7:00 – 9:00)

[Action: Select preset **DEMO-AUTO-001** or paste Subject: 'Resolving invalid credential errors on console login', Customer: 'Priya Sharma'. Click 'Process Ticket Through Pipeline']

"First, let's process ticket ID **DEMO-AUTO-001** submitted by Priya Sharma regarding an invalid credential login error.

I click **Process Ticket Through Pipeline**.

Notice the pipeline execution results:

- **Action Taken:** Updates to **AUTO_RESPOND** in green.
- **Predicted Intent:** `authentication_failure` with an exact confidence score of **zero point ninety-two** (0.92).
- **Response Tab:** Gemini generates a grounded solution citing `[DOC-AUTH-001]`.
- **FAISS Tab:** Expanding Passage 1 shows chunk score 0.88 extracted from **DOC-AUTH-001** titled 'Resolving invalid credential errors on login'."*

Demo Case 2: Deterministic Safety Escalation (**DEMO-ESC-002**) (9:00 – 11:00)

[Action: Select preset or paste Subject: 'Dispute regarding overcharge on June billing invoice', Body: 'Our invoice for June shows a charge of \$4,500... Please issue a refund...'. Click 'Process Ticket Through Pipeline']

"Now let's process ticket ID **DEMO-ESC-002** submitted by Carlos Gomez regarding a billing overcharge dispute of \$4,500.

I submit the ticket. Notice that even though the classification confidence is zero point ninety-five, the intent `billing_dispute` triggers our mandatory safety override rule (`must_not_auto_respond = True`).

The status badge immediately updates to **ESCALATED** in yellow, displaying the reason: 'Policy requirement: `billing_dispute` issues must be reviewed by human support', while attaching the relevant documentation context for senior engineers."

Demo Case 3: Guardrail Hard Intercept (**DEMO-PII-003**) (11:00 – 12:30)

[Action: Select preset or paste Live Chat Body: 'My deployment build fails with auth error. Here is the production key I am using: `sk_live_998877665544332211`. Please check...'. Click 'Process Ticket Through Pipeline']

"Next, let's observe our safety execution guardrails in action on ticket ID **DEMO-PII-003**.

The customer pasted a live secret API key starting with `sk_live_998877...` into live chat.

I click submit. The regex guardrail in `src/guardrails.py` intercepts the text, flags a PII violation: 'Secret API Key detected', and **HARD BLOCKS** the response.

Under the Guardrails tab, status displays **BLOCKED** in red, guaranteeing zero confidential data leaks."

Demo Case 4: Unattended Evaluation Harness Run (**DEMO-EVAL-004**) (12:30 – 14:00)

[Action: Switch to Terminal Window. Run command: `python -m evaluation.harness --input FDE_Capstone_Docs/05_Datasets/validation_tickets.json --output evaluation/results/`]

"Finally, let's run our unattended evaluation harness against the eighty validation tickets in `validation_tickets.json`.

I execute: `python -m evaluation.harness --input FDE_Capstone_Docs/05_Datasets/validation_tickets.json --output evaluation/results/`.

*The harness processes all **eighty tickets unattended** in just **zero point forty-two seconds** without manual intervention.*

*Looking at the terminal output: Eighty out of eighty decisions are written to `storage/decisions.db`, achieving a 95th percentile latency of **zero point zero one zero four seconds** (0.0104s), and the metrics report is saved to `evaluation/results/metrics_report.json`."*

Section 5: Results & Metrics (14:00 – 17:00)

[Visual: Screen Share – Streamlit Metrics Dashboard & JSON Report]

"Let's examine what the evaluation metrics demonstrate:

>

*In terms of **Business Outcomes**, on documentation-grounded tickets, our First Contact Resolution reached **sixty-eight point five percent**, surpassing our sixty-five percent target. First-reply response latency dropped from eight hours to a 95th percentile of **zero point zero one zero four seconds** on the harness and under two point five seconds on live API calls.*

>

*In terms of **Technical Performance**, intent classification accuracy achieved high precision across core categories, and citation accuracy reached ninety-two percent.*

>

*In terms of **Governance**, logged decision coverage is exactly one hundred percent with eighty decisions reconciled in `decisions.db`. Zero private data leaks occurred across all test runs."*

Section 6: Governance & Risk Management (17:00 – 18:00)

[Visual: Screen Share – Governance Tab / Streamlit Kill Switch]

"Governance was designed into the architecture from day one.

>

Our risk register mitigates PII leakage via regex guardrails, eliminates hallucinations through mandatory document grounding, and prevents improper financial commitments.

>

In our fairness audit, citation accuracy across non-fluent vs fluent language groups varied by less than three point two percentage points.

>

*Furthermore, we built an instant **Emergency Kill Switch**. If I toggle the Kill Switch on in our control panel, environment variable `KILL_SWITCH_ACTIVE` updates to true, instantly shifting the entire application into 100% escalation mode until resolved."*

Section 7: PRD Revisions & Conclusion (18:00 – 20:00)

[Visual: Camera On – Sourav Nayak on screen]

"During contact with real data, we revised our PRD twice. First, we calibrated our confidence threshold from zero point eighty-five to zero point eighty after observing that zero point eighty-five caused an unnecessarily high escalation rate without increasing precision. Second, we converted PII guardrails from warning logs to hard blocking.

>

If we had more time, our next step would be implementing dynamic hybrid search combining BM25 keyword search with FAISS dense vector retrieval to handle exact code snippet matching even better.

>

In summary, by shifting focus from a simple chatbot to an intelligent automated knowledge delivery and escalation engine, we met all twelve acceptance criteria, cleared the unattended gate, and provided CloudServe with a trusted system. Thank you."